

Scripts d'extractions et de traitement de données de vie réelle d'ARIA à des fins de recherche clinique

ARIA ODM Builder

Notice visuelle du dépôt GitHub



Comment lire ce document

But du PDF

Ce document est conçu pour être lu directement depuis le dossier docs/ du dépôt GitHub.

- ▶ comprendre l'ordre réel du pipeline ;
- ▶ identifier les fichiers importants ;
- ▶ savoir quel script sert à quoi ;
- ▶ comprendre le résultat attendu.

Public visé

Utilisateur non spécialiste du code : médecin, physicien médical, interne, chercheur ou collaborateur projet.

Ce PDF fonctionne comme un README visuel.

Transformer des extractions ARIA en fichiers patients structurés, contrôlés et exploitables pour la recherche clinique.

Extraire

Récupérer les informations utiles avec les requêtes SQL.

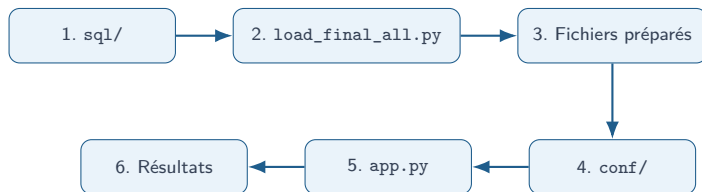
Préparer

Nettoyer et harmoniser les données avec le script principal.

Exporter

Produire un résultat structuré avec l'application Streamlit.

Chaîne complète du projet



- ▶ `sql/` décrit l'extraction des données sources.
- ▶ `scripts/load_final_all.py` sert de script principal de préparation et de nettoyage.
- ▶ `app.py` est ensuite utilisé pour charger, contrôler et exporter les données via Streamlit.

Organisation visible sur GitHub

```
ARIA_ODM_Builder/  
|-- conf/  
|-- docs/  
|-- pictures/  
|-- scripts/  
|-- sql/  
|-- utils/  
|-- .gitignore  
|-- .pre-commit-config.yaml  
|-- README.md  
|-- app.py  
'-- requirements.txt
```

Lecture rapide

Le dépôt est organisé pour suivre la chaîne :
extraction SQL, préparation par scripts,
configuration, application Streamlit et export.

Point d'entrée utilisateur

Pour l'utilisation guidée, l'utilisateur lance
app.py. Pour la préparation amont, le script
principal est dans scripts/.

Étape 1 : requêtes SQL dans sql/

```
sql/  
|-- fichier_A_patient.sql  
|-- fichier_B_patient.sql  
|-- fichier_C_patient.sql  
'-- README.md
```

Rôle

Produire les premières extractions nécessaires au pipeline.

- ▶ extraction des données de traitement ;
- ▶ extraction des formulaires cliniques ;
- ▶ extraction d'une source complémentaire si nécessaire.

Cette étape est réalisée par l'utilisateur disposant des accès nécessaires.

Connexion à la base ARIA : où renseigner les paramètres

Où modifier ?

Les informations de connexion sont à renseigner dans le script principal de préparation :

```
scripts/load_final_all.py  
fonction : connecter_bdd()  
chercher : pyodbc.connect(...)
```

- ▶ nom du serveur ;
- ▶ nom de la base ;
- ▶ mode d'authentification ;
- ▶ port si nécessaire.

Exemple à adapter

```
pyodbc.connect(  
    "Driver={ODBC Driver 17 for SQL Server};"  
    "Server=REEMPLIR_ICI_NOM_SERVEUR,PORT;"  
    "Database=REEMPLIR_ICI_NOM_BASE;"  
    "Trusted_Connection=yes;"  
    "TrustServerCertificate=yes;"  
)
```

À retenir : l'utilisateur complète ce bloc avec les paramètres fournis par son environnement de travail.

Étape 2 : préparation dans scripts/

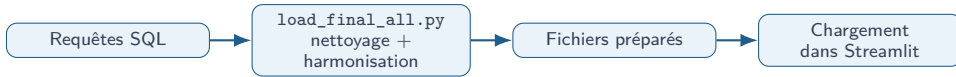
```
scripts/  
|-- README.md  
|-- load_final_all.py  
|-- sys.py  
|-- tox.py  
'-- trait.py
```

Script principal

load_final_all.py centralise l'exécution
amont : chargement, harmonisation,
nettoyage et génération de fichiers préparés.

- ▶ trait.py : logique liée aux traitements ;
- ▶ tox.py : logique liée aux toxicités ;
- ▶ sys.py : fonctions système ou utilitaires.

Sortie de la préparation amont



Fichiers obtenus

Les fichiers produits servent d'entrées au workflow Streamlit.

Intérêt

Cette étape évite de mélanger extraction, nettoyage et utilisation interactive dans une seule partie du code.

Étape 3 : fichiers d'entrée pour Streamlit

```
inputs/  
|-- source_A.csv  
|-- source_B.csv  
'-- source_C.csv    # optionnel
```

source A

fichier de traitement préparé

source B

fichier de formulaires préparé

source C

fichier complémentaire optionnel

Ces fichiers sont ceux que l'utilisateur charge ensuite dans l'interface Streamlit.

Étape 4 : configuration dans conf/

`mapping.csv`

Indique quelle colonne source correspond à quelle variable utilisée dans le pipeline.

Exemples : identifiant patient, date, dose, diagnostic, fractionnement.

`profiles/*.json`

Décrit un scénario d'extraction : pathologie, critères, variables attendues et règles de sélection.

Exemples : codes CIM10, variables cliniques, règles de cohorte.

Le mapping dit où lire. Le profil dit quoi sélectionner.

Installer le projet

Étape utilisateur

Créer un environnement Python puis installer les dépendances.

```
git clone <URL_DU_DEPOT>
cd ARIA_ODM_Builder
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

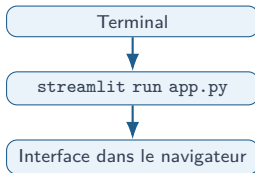
- ▶ une seule installation au départ ;
- ▶ les dépendances sont listées dans `requirements.txt` ;
- ▶ l'environnement évite de mélanger les projets Python.

Étape 5 : lancer l'application Streamlit

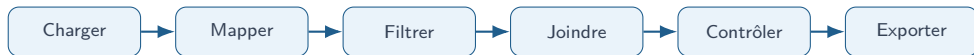
Commande

```
streamlit run app.py
```

- ▶ l'application s'ouvre dans le navigateur ;
- ▶ l'utilisateur charge les fichiers préparés ;
- ▶ le pipeline peut être recalculé depuis l'interface.



Ce que fait app.py



- ▶ interface principale du dépôt ;
- ▶ guide l'utilisateur étape par étape ;
- ▶ s'appuie sur les modules du dossier `utils/`.

À retenir

L'utilisateur standard lance `app.py` après la préparation des fichiers d'entrée.

Modules Python utilisés par l'application : utils/

Module	Rôle
load.py	charger les fichiers d'entrée
clean.py	nettoyer et standardiser les données utiles
mapping.py	appliquer les correspondances de variables
cohort.py	construire la cohorte selon les critères choisis
temporal.py	calculer les variables liées au temps
quality.py	produire les contrôles qualité
export.py	générer les fichiers de sortie
profile.py	lire et valider les profils JSON
display.py	afficher les résultats dans Streamlit
text.py	gérer les textes, libellés et messages

Résultats attendus après exécution

Fichiers produits

- ▶ export patient structuré ;
- ▶ rapport qualité ;
- ▶ trace d'exécution.

Utilisation

Ces sorties servent de base au contrôle, à la relecture puis aux analyses de recherche clinique.

- ▶ vérifier la cohorte ;
- ▶ relire les alertes qualité ;
- ▶ exploiter l'export final.

L'utilisateur sait alors quels fichiers ont été utilisés et quel résultat a été généré.

Workflow résumé



- ❶ Exécuter les requêtes du dossier `sql/`.
- ❷ Préparer et nettoyer avec `scripts/load_final_all.py`.
- ❸ Charger les fichiers préparés dans Streamlit.
- ❹ Choisir le mapping et le profil dans `conf/`.
- ❺ Lancer les contrôles qualité.
- ❻ Générer l'export final.